

VelaGuard 技术报告

2026 首届 openvela AI 硬件开发者大赛

目 录

- 1、信息表..... 1
- 2、摘要..... 1
- 3、正文..... 2
 - 3.1 绪论..... 2
 - 项目背景与问题定义..... 2
 - 技术难点..... 2
 - 创新点..... 3
 - 3.2 系统方案设计..... 3
 - 系统总体架构..... 3
 - 方案论证与选型..... 4
 - 关键模块设计..... 5
 - 3.3 核心算法与技术原理..... 6
 - 云端推理与 ReAct 工具轮次..... 6
 - 确定性规则与确认机制..... 7
 - openvela 能力与资源组织..... 8
 - 3.4 系统实现..... 8
 - 软件架构与启动..... 8
 - 四条数据路径..... 10
 - 硬件设计与适配..... 11
 - 交互与设备 Skill..... 13
 - 配套项目：上位机与云看板..... 15
 - 复现入口..... 16
 - 3.5 系统测试与结果分析..... 17
 - 测试环境与证据范围..... 17
 - 带屏静止首页性能..... 18
 - Agent 时延与已有镜像..... 18
 - 可靠性与稳定性..... 19
 - 3.6 AI-Native 开发说明..... 19
 - 开发方式与经验..... 20
 - 3.7 总结与展望..... 21
 - 成果总结..... 21
 - 应用前景与商业价值..... 21
 - 不足与后续重点..... 22
 - 实现与证据索引..... 22

1、信息表

项目	内容
作品名称	VelaGuard
队伍名称	Team Falcons
团队分工	叶培林负责固件、扩展板方案、LVGL HMI、ai_agent 集成、文档与公共仓 Pull Request
选题方向	AI 硬件产品创新

2、摘要

VelaGuard 是 STM32H750B-DK 上的 Modbus 现场值守网关，同板用 openvela 运行 LVGL 界面与 ai_agent。采集、判定、告警与显示由本地 C 代码完成，断网照常；联网后 Agent 按 Skill 解释告警，并经 MQTT 四类主题上报云看板。重点是本地安全环、试读后人工确认、受限 Agent 与 AI 推测标注，另有 Windows 上位机与只读云看板配套。已提交公共仓 PR 8 个；2026-09-13 带屏静止首页约 80 秒的提交由 158 次降至 9 次；2026-08-30 无屏板端一次日报 115 秒，完成 6 轮 6 次工具调用。

资料核对日期为 2026-09-15。专属仓为 [contest2026_004_TeamFalcons](#)

3、正文

3.1 绪论

项目背景与问题定义

目标用户是把设备接到陌生 485 总线上的嵌入式工程师与系统集成人员。现场痛点不是缺一个问答窗口，而是缺一个能独立采集、能当场看、配置改错也回得去的本地节点。现场值守人员要一直看到温度、水浸等设备的真实状态，串口调试工具能读寄存器，却要电脑一直连着；再加一个云端问答，也替代不了本地采集、告警和配置管理。VelaGuard 把这些事集中到一台带屏现场网关，配置、运行和解释各自有清楚的入口。

集成人员接入 Modbus 从站，经调试串口或上位机编辑候选点表，试读并检查实际值后人工确认；网关随后独立采集并显示。发生越限或离线时，规则先给出可追溯的告警，联网助手再帮助值守人员理解数据。Modbus 采用主站请求、从站回应的寄存器访问方式。[\[1\]\[2\]\[10\]](#)

技术难点

同一颗 Cortex-M7 处理器内核要同时承担图形刷新、总线采集、网络通信、TLS 加密连接和 Agent 工具轮次。项目要协调栈和缓冲区预算，避免文件读写占用 UI 线程，并让图形更新只处理真正变化的内容。QSPI 外部闪存中执行的应用、eMMC 存储中的数据和片内启动代码也要配合；RJ45 与 ESP-01S 的网络管理则要和本地业务分开。[\[3\]\[6\]\[19\]](#)

创新点

采集、阈值与离线判定、告警状态和显示路径都由确定性 C 代码负责，形成本地安全环。点表与实际读数决定告警，模型回答不参与触发或恢复。[\[1\]\[3\]](#)

候选点表与采集点表分开，编辑和试读不会直接更换运行中的点表。操作员看到试读结果后，单独执行带 `--confirm` 的生效命令。[\[2\]\[10\]](#)

运营助手只做查询和解释，配置命令在 C 层受限。模型解释会标为 AI 推测，与本地规则结论分开。已落实的工具层限制见图 3，待补强项见 3.7。[\[4\]\[19\]](#)

板端 Agent、Windows 点表上位机与只读云看板构成端端与端云分工：电脑负责生成和确认配置，板端负责本地判断与解释，云端只做留档与检索，三者通过串口协议与 MQTT 四类主题解耦。[\[28\]\[30\]](#)

3.2 系统方案设计

系统总体架构

系统由现场 Modbus 从站、H750B-DK 网关和联网服务组成。现场从站负责寄存器数据，openvela 网关负责主站采集、配置、图形和 Agent 运行；MiMo 提供云端推理，MQTT Broker 接收设备上报。LVGL 是嵌入式图形组件库，ai_agent 是 openvela 的设备端助手框架。电脑用于上位机配置、开发和救援；云端另有只读看板，订阅四类主题并留存历史，不向设备下发命令。

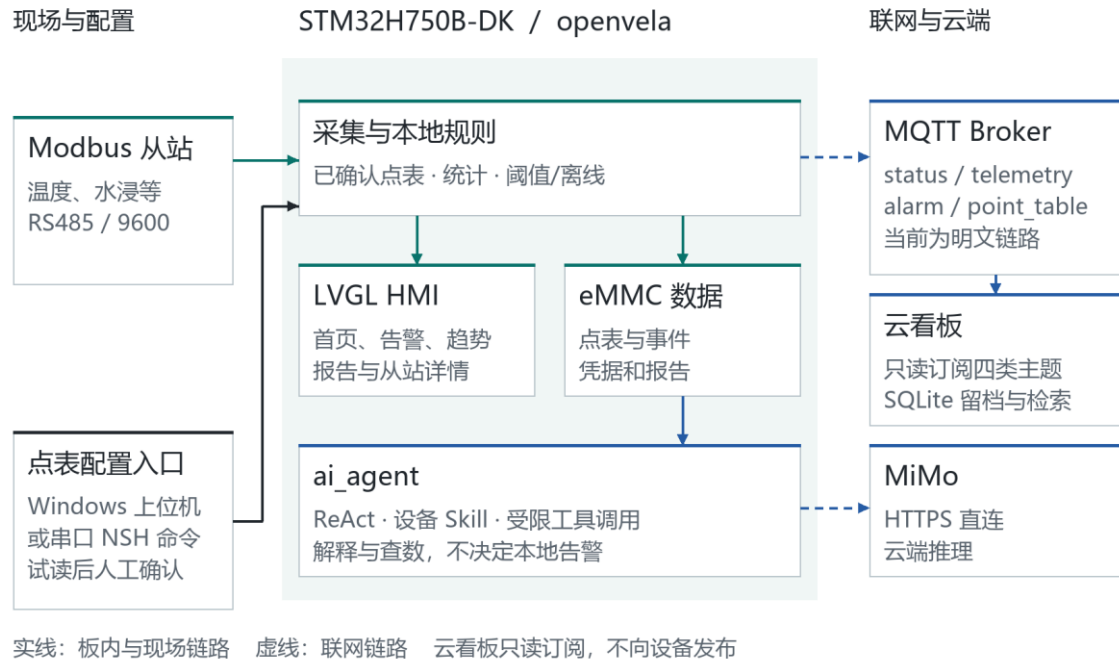


图 1 系统总体架构：现场、板端与云端的职责分工

方案论证与选型

项目选用 openvela 已支持的 STM32H750B-DK，板载显示、触摸、QSPI、SDRAM、eMMC 和以太网接口都能直接用。另加一台长期在线电脑，会把采集与现场显示继续交给主机；把屏幕和网关拆成多个节点，又会多出同步和部署环节。同板实现少了这些问题，但内存预算更紧。[\[9\]\[19\]](#)

ai_agent 的 llm_proxy 发起大模型请求，通过 OpenAI 兼容 HTTPS 接口直连 MiMo。设备凭据由 vgprovision 管理并加密保存在 eMMC，产品固件不内置可用 API key。该选择使 LLM 请求不依赖另建的 MQTT AI 转发服务。[\[7\]\[19\]](#)

本地有周期采集、规则告警、配置读写、运行统计和 LVGL 显示。断网时这些路径不依赖模型；告警解释和自然语言查询才需要 MiMo 在线。屏幕上的运行报告由固件统计，联网生成的解释与它分开。[\[1\]\[3\]\[5\]](#)

联网侧不再自建 AI 转发服务：板端 Agent 经 HTTPS 直连 MiMo，MQTT 只承载状态与数据，供只读看板消费。上位机做成 Windows 图形程序，直接使用板端已有的 `vgpoint` 协议，不为每台现场笔记本再部署运行时。三个部分各自可单独替换，也不增加板端的内存负担。[28][30]

关键模块设计

表 1 模块职责与输入输出

模块	职责	主要输入与输出
采集与点表	RS485 主站读取已确认点表	寄存器读数、有效性、最近更新时间
规则与统计	阈值、离线、帧质量和运行累计	告警状态、异常记录、pending 文件
LVGL HMI	现场查看与逐点操作	首页、告警、趋势、报告、从站详情
ai_agent	按 Skill 调用工具并组织解释	问题或待处理告警 → 工具证据 → 文本
网络与 MQTT	单活动链路管理及设备上报	status、telemetry、alarm、point_table
上位机（配套）	编辑与导入候选点表，试读后确认	vgpoint 协议命令、实时读数快照
云看板（配套）	只读订阅四类主题并留档检索	SQLite 历史、中文网页查询

RJ45 是主链路，ESP-01S 备用，固件维护连接状态、健康检查和失败退避。MQTT 走明文测试链路，只承载状态与数据，LLM 请求另走 HTTPS 直连。2026-09-14 板端验收四项断言全部通过（PASS=4 FAIL=0），涵盖设备标识、链路状态、以太网地址与 mqtt 在线。[6][8][20][21]



图 2 板端首页：32 个点位的快照与状态栏

首页按告警、离线、正常三态过滤点位，顶栏汇总网络、采集与告警状态。点表上限为 32 个点位、32 个扫描地址；单从站帧统计保留 8 个窗口桶。上图由无头用例驱动真实 LVGL 代码渲染，板端屏幕分辨率为 480×272。[25][14]

3.3 核心算法与技术原理

云端推理与 ReAct 工具轮次

推理在云端完成，设备端负责会话、Skill 加载与工具调用，本节的量化集中在接口、轮次与工具次数。MiMo 通过 /v1/chat/completions 接口和 Bearer 鉴权提供推理；ai_agent 在板上维护会话、加载 Skill，并运行 ReAct 循环。Skill 是约定任务步骤与约束的 Markdown 文件。ReAct 指模型根据工具返回的结果继续推理，直到形成回答或达到轮次限制。[7][19]

以告警解释为例，Skill 规定先读取 pending_alarm.txt，再查询帧统计、寄存器和配置摘要，随后把说明写入 last_alarm.md。数据缺失必须保留

不确定性，不能用推测补齐真实读数。工具调用与拦截通过框架的系统日志记录。[4][19]

确定性规则与确认机制

阈值规则支持 `ge`、`le`、`eq`，即大于等于、小于等于和等值比较，并区分 `warn` 预警与 `crit` 严重告警。等值比较使用 `0.0005` 的数值容差。读失败进入离线判断；板端还会检查连续失败次数和最近 8 轮的失败窗口，窗口默认至少 5 次失败，并结合 `fail_n` 配置过滤短时抖动。多个告警并存时，严重、离线、预警按顺序选择主要告警。[1][3]

候选点表先经过字段检查和试读，操作者查看结果后再单独确认生效。`vgpoint apply` 的 C 实现要求 `--confirm`，编辑和试读命令不调用这一提交路径。试读记录与生效版本的强绑定尚未实现，列入 3.7。[2][10]

操作员的配置流程



设备端 Agent / 当前 C 实现

入口	已存在的限制	适用边界
<code>vgpoint / vgdiscover</code>	<code>run_shell</code> 拒绝调用整个命令	不开放配置入口
<code>vgcfg</code>	仅放行 <code>dump</code> 查询	限制该命令
文件工具	限定数据目录；保护部分配置文件	仍需收紧写路径
<code>vgstats / vgnet</code>	命令在允许表内	子命令限制待补强

产品目标：只读查询、AI 推测与规则结论分开。

允许表不等于完整只读证明；子命令限制与试读绑定待补强，见 3.7。

图 3 本地安全环与点表人工确认闸门

openvela 能力与资源组织

图形部分由 LVGL 页面、模型快照和显示触摸驱动配合完成。采集结果通过受保护的快照交给 UI，报告读写和 `pending` 文件操作由后台工作线程处理。页面按变化刷新，保留固定对象和监听器，减少静止页面的重复提交。趋势页保留单点历史曲线与阈值线，板端历史深度 128 点，可切换最近 60 点窗口，越限点单独着色。[3][13][14][25]

对照赛道要求，本项目落地了 `openvela` 系统能力中的两项。图形部分使用 `openvela` 的 LVGL 图形栈、显示与触摸驱动，以及自绘的现场页面；AI 部分使用 `openvela` 的 `ai_agent` 框架，包括 `ReAct` 循环、`Markdown Skill`、`HEARTBEAT` 主动任务与 `NSH` 交互渠道。多媒体能力本作品未使用。[19]

AI 能力使用 `openvela` 的 `ai_agent`、`llm_proxy`、`Skill` 加载、心跳服务和 `NSH` 交互渠道。带屏配置将 `Agent` 主循环栈设为 32 KiB、上下文和流缓冲各设为 4 KiB，并按需启动主循环。这些数值说明配置预算，整机余量见 3.7。[19]

公共仓里的改动涉及 `QSPI` 启动、板级接口、显示、触摸、网络与 `Agent` 适配。后续可以把受限 MCU 的栈和缓冲预算整理成配置组合，并加强工具参数、写路径和输出结构的检查。8 个 PR 的具体入口见表 4。

3.4 系统实现

软件架构与启动

`velaguard_app_main` 托管 `NSH` 与产品业务，初始化配置和网络管理，安装设备 `Skill`，启动 HMI 后延迟启动 `ai_agent --daemon`。`daemon` 是后台常驻服务，`NSH` 是设备的命令行终端，可用 `ai_agent` 命令进入交互渠道。

带屏自启已在历史启动记录中观察到，完整工具轮次的验证范围见 3.5。

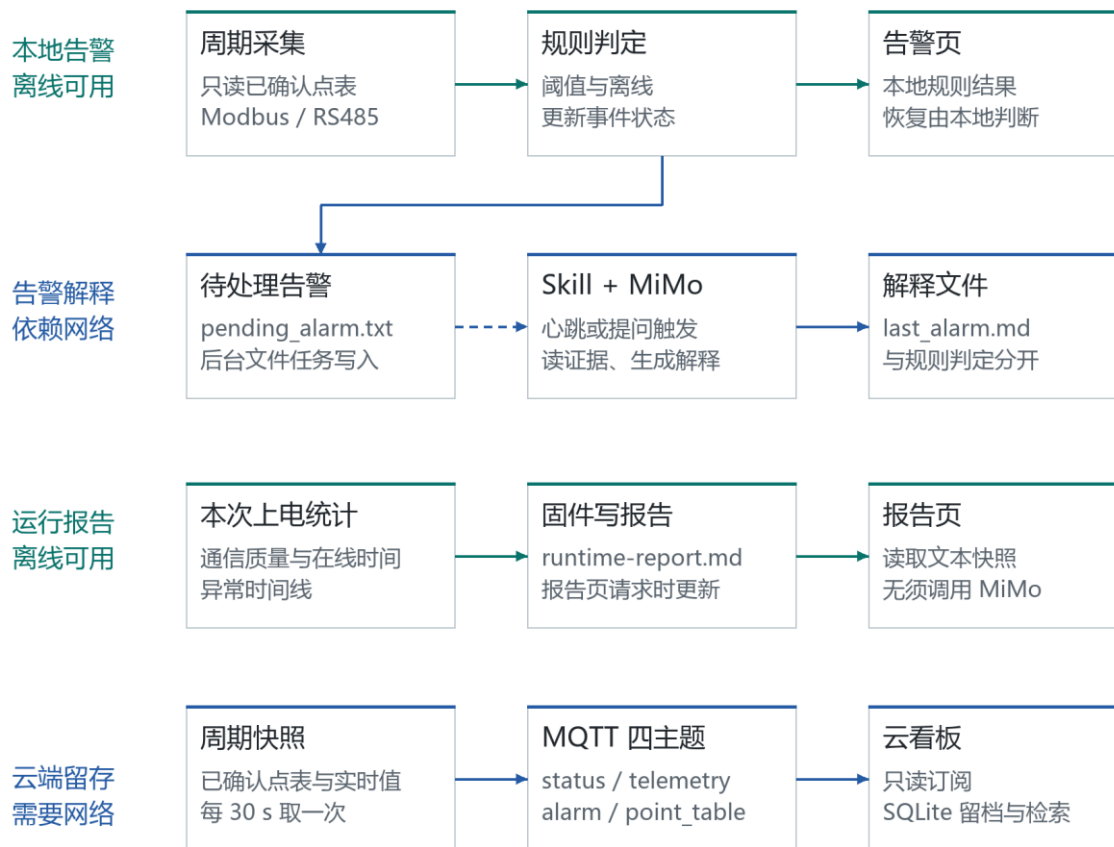
[3][4][11]

表 2 固件组成

层次	实现与职责
交互层	LVGL 首页、告警、趋势、运行报告和探查页面；NSH 配置与查询命令
业务层	点表、Modbus 采集、帧统计、告警判定、运行统计、后台文件处理
AI 层	ai_agent 会话和 ReAct；Markdown Skill；llm_proxy 网络推理
系统与驱动	openvela / NuttX 任务和文件系统；串口、网络、显示、触摸、QSPI 与 eMMC

四条数据路径

不同产物使用不同路径



四类主题由网络管理线程发布；采集与界面线程只入队，队列满时丢弃 telemetry、保留 alarm。

图 4 本地告警、联网解释、运行报告与云端留存的数据路径

周期采集读取已确认点表，规则计算当前状态，形成本地告警；HMI 展示告警，后台任务写入主要告警对应的 pending_alarm.txt。告警恢复后，由本地逻辑清理待处理文件并记录状态变化。[\[1\]\[3\]](#)

HEARTBEAT.md 规定发现 pending 后调用告警 Skill；Agent 取证并输出 last_alarm.md。心跳触发与手动 ask 是两种入口，这条事件主动链路的板端完整验收见 3.5。[\[4\]\[12\]](#)

vg_runtime 累计本次上电以来的通信质量、点位在线时间和异常记录。报告页请求读取或刷新时，后台任务写入 runtime-report.md，再返回

文本快照。operations_report Skill 读取并复述这份内容，不负责覆盖它。

[3][4][5]

已确认点表的周期快照按 status、telemetry、alarm、point_table 四类主题上报，由网络管理线程统一发布；采集与界面线程只入队，队列满时丢弃 telemetry、保留 alarm。云看板只读订阅这四类主题，写入 SQLite 后供网页查询。[8][20][28][29]

硬件设计与适配

本项目不做全新硬件平台适配，进行了驱动开发。主板是 openvela 已支持的 STM32H750B-DK，扩展方案通过 Arduino 与 STMod+ 接入 RS485 和 ESP-01S。接口映射已与 MB1381-B01 原理图第 14、16 页及当前 board.h 交叉核对；下表给出方案级关键器件。[9]

表 3 接口与关键器件

接口或模块	接线与实现
RS485	UART7 使用 PB4/D10 作为 TX，PA8/D5 作为 RX，PK1/D4 控制方向；3.3 V 收发器可选 SP3485/MAX3485 类，配终端电阻与总线保护。
备用 Wi-Fi	ESP-01S 经 USART2 的 PD5/PD6 接 STMod+，需对应焊桥选择与独立 3.3 V 供电。
显示与存储	使用主板 LCD/触摸、QSPI、SDRAM 与 eMMC；应用经 QSPI XIP 执行，配置和报告在 eMMC。
调试与输出	ST-LINK 虚拟串口用于 NSH，不占用 RS485；扩展方案含低边 MOSFET 数字量输出，设备 Agent 不负责控制。

扩展板接口与接线 (Arduino / STMod+ 扩展方案)

STM32H750B-DK 主板
LCD 480×272 触摸 · QSPI XIP 应用 · SDRAM 帧缓冲 · eMMC 配置与报告 · RJ45 以太网

RS485 半双工
UART7: PB4/D10 发送
PA8/D5 接收, PK1/D4 方向
3.3 V 收发器与终端电阻

备用 Wi-Fi
ESP-01S 经 USART2
PD5 / PD6 接 STMod+
焊桥选择与独立 3.3 V

调试与输出
ST-LINK 虚拟串口
NSH, 不占用 RS485
低边 MOSFET 输出

RS485 方向由驱动在发送完成后切回接收; 帧间隔只保留 RTU 的 3.5 字符。
接口映射已与 MB1381-B01 原理图第 14、16 页及当前 board.h 交叉核对。
ESP-01S 备用链路的代码路径已实现; 2026-09-14 板测记录中 Wi-Fi 未关联, 切换行为另测。

图 5 扩展板接口与接线

XIP 指程序直接从外部闪存取指执行, QSPI 启动工作解决片内启动代码与外部执行镜像的衔接; 显示与触摸改动分别进入 LTDC 和 FT5x06 驱动, ESP8266 与 MQTT 改动留在其真实公共源码树中。以下 8 个 PR 的目标分支均为 dev-ai-contest-2026, 2026-09-15 经 GitHub API 核实状态为 open。[11]

表 4 公共仓贡献入口

公共仓 PR	主要内容
nuttx #350	片内 boot stub 与 QSPI XIP 启动支持
nuttx #351	VelaGuard 扩展板接口与板级配置
nuttx #352	网络、eMMC、RS485 和 ETH MII 的板级 bring-up
nuttx #353	STM32H7 LTDC 显示加速
nuttx #354	FT5x06 触摸轮询性能改进
nuttx-apps #119	链路状态轮询、DHCP 续租及 ESP8266 LESP 兼容
MQTT-C #1	带标记的 LESP send/recv 弱符号钩子
ai_agent #32	H750 带屏内存配置与 VelaGuard 工具访问限制

交互与设备 Skill

现场有三个交互入口：LVGL 触摸界面负责查看与逐点操作；NSH 提供候选点表编辑、试读、确认和实时值查询，在串口终端输入 `ai_agent` 即可进入 `vela>` 对话渠道；Windows 上位机把同一条串口流程做成图形界面。RS485 总线只由网关作为主站使用。配置完成后不要求电脑长期连接；自动扫描由确定性代码负责，界面扫描需由用户显式开启，Agent 不参与扫描或提交配置。[\[2\]](#)[\[10\]](#)[\[30\]](#)



图 6 告警页：三行活动告警与逐点静音、标记处理



图 7 趋势页：阈值线与历史窗口切换

表 5 三份设备 Skill

文件	触发与数据	输出
alarm_interpretation.md	告警提问或待处理告警；读取 pending、统计、寄存器和配置摘要	解释文本写入 last_alarm.md
operations_report.md	用户询问运行概况；读取固件生成的 runtime-report.md	口述数据与异常，不覆盖原报告
modbus_query.md	用户查询寄存器、通信质量或运行累计；调用查询工具	返回工具支持的数据

三份 Skill 均为 Markdown 文件，由固件首次启动写入 /data/agent/skills/，约定任务过程，C 层工具实现负责真正的访问限制。自然语言查数是补充交互，使用演示见提交的演示视频。[\[4\]\[19\]](#)

配套项目：上位机与云看板

作品本体是专属仓内的固件；上位机与云看板是配套工具，独立成仓便于单独部署与替换，合起来构成设备、电脑、云端三端分工。

[10][28][30]

点表上位机（velaguard_host）是 Windows 图形程序，讲的是板端既有的 vgpoint NSH 协议：新增、导入、删除点位，随后试读候选值，操作员确认后落盘。落盘仍由板端 C 实现执行，仍要求 `--confirm`；上位机只是把同一条串口流程做成界面，「确认落盘」在界面上单独标为醒目操作。[10][30]

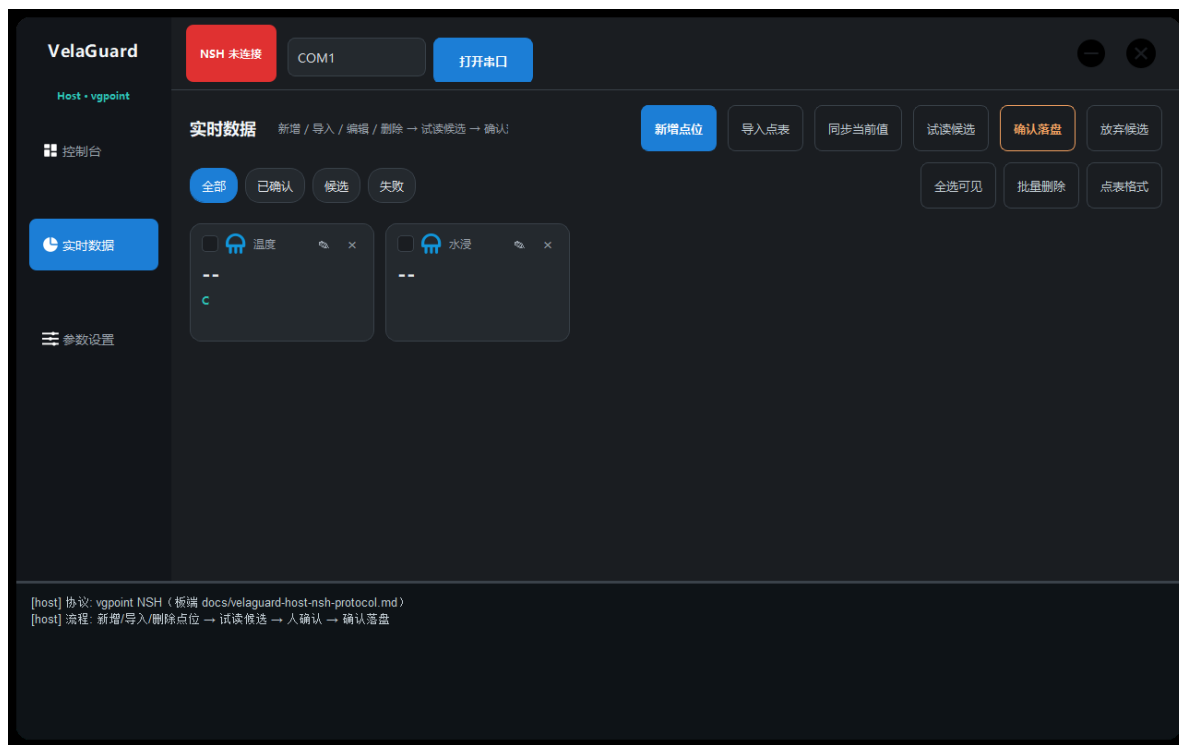


图 8 点表配置上位机：编辑、试读与人工确认；点表卡片取自示例文件

云看板（velaguard_mimo2mqtt）是独立部署的只读服务，在线地址为 velaguard.19y.cc。采集器订阅 `vg/{device_id}/status`、`telemetry`、`alarm`、`point_table` 四类主题，写入 SQLite，网页端用中文界面查询设备总览、

实时值、时序趋势与告警历史。服务只提供 GET 接口，从不向设备主题发布，也不清告警；入口为 `vg-dashboard`。[20][28][29]

两个配套项目与板端的分工是单向的：电脑能改配置，云端只能看。板端 Agent 既不改配置，也不写云端。[4][28]

复现入口

需要包含 `nuttx`、`apps` 和 `packages` 的完整 `openvela` 工作区，并按专属仓 `manifest` 接入产品目录。在本仓根目录执行 `bash scripts/build.sh`，默认构建 `velaguard-lvgl`。公共树改动按表 4 核对；本仓构建脚本不会替公共树应用补丁。主机测试入口为 `make -C app/velaguard/host_tests test`，当前 12 个用例覆盖网络策略、配置存储、帧统计、运行统计、探查、点表、告警判定、时间同步、HMI 性能、运行渲染、交互调度与 MQTT 载荷；无头界面用例用 `ctest` 运行，覆盖告警、趋势、夹具、生命周期刷新与交互检查。上位机与云看板各自按仓内 `README` 运行。本仓源码遵循 Apache 2.0，内置的 `nanoMODBUS` 保留其 MIT 许可。[16][23][30]

3.5 系统测试与结果分析

测试环境与证据范围

带屏性能记录来自 2026-09-13 的 STM32H750B-DK，通过 `bash scripts/build.sh --hmi-perf` 构建测量固件，首页装载 14 个已确认点位，采集线程运行、无触摸。记录中 `live ok=0/14`，当时无成功实读。Agent 日报记录另来自 2026-08-30 的历史无屏固件，两组结果不混为同一固件验收。[\[11\]](#)[\[12\]](#)[\[13\]](#)

下表保留原测试日期；本轮只核对源码与原始记录，未重新烧录。

表 6 功能测试与实现核查

项目与日期	方法或条件	实际结果与证据
主机测试 2026-09-15 复核	<code>app/velaguard/host_tests</code>	12 个用例编译运行通过，覆盖网络策略、配置存储、帧统计、运行统计、探查、点表、告警判定、时间同步、HMI 性能、运行渲染、交互调度与 MQTT 载荷。 [23]
无头界面测试 2026-09-13 18:28	<code>ctest, hmi-headless</code>	原始记录 5/5 通过，覆盖告警、趋势、夹具、生命周期刷新和交互检查；不能代替实体触摸验收。 [11]
MQTT 四主题板验 2026-09-14 11:14	板端串口脚本断言	PASS=4 FAIL=0：设备标识由 UID 派生、 <code>vgnet</code> 显示 <code>mqtt=up</code> 、 <code>eth0</code> 取得 192.168.137.47、运行日志出现 <code>mqtt online</code> 。 [20] [21]
板端时间同步 2026-09-14	串口连续观察记录	16:16 至 21:17 约 5 小时， <code>vgtime</code> 校时成功 11 次，同段日志无断言失败或 <code>panic</code> 。 [22]
带屏 Agent 自启 2026-09-13	启动串口日志	出现 <code>ai_agent autostart ok</code> 、32768 字节栈和 <code>daemon</code> 启动信息；只证明服务自启。 [11]
历史 Agent 日报 2026-08-30	无屏板端，手动 ask	END status=ok；6 轮、6 次工具调用，产生 1796 B 日报文件。 [11] [12] [31]

项目与日期	方法或条件	实际结果与证据
点表确认与本地告警 2026-09-14 核查	代码、协议及历史用例	候选分离与 --confirm 检查存在，点表上限 32 点；完整人工试读确认与物理注入录像仍待补充。[1][2][10]
带屏工具轮次 2026-09-14	串口日志与故障转储	含工具轮次曾出现 ai_agent 任务 HardFault (mm_forcefree, CFSR=00000400)，最后一次记录在 15:19；其后观察无复位。[3][15][24]

带屏静止首页性能

表 7 同条件前后快照

指标	优化前	优化后
采样时刻	uptime 81.350 s	uptime 80.430 s
界面提交 commits	158 次	9 次
render 次数 / 最大值	81 次 / 140 ms	9 次 / 140 ms
首次导航最大值	190 ms	200 ms
页面对象 / 监听器 / 定时器	71 / 2 / 6	71 / 2 / 6
观测器 dropped	0	0

约 80 秒窗口中，提交次数明显下降，而页面对象、监听器和定时器计数保持不变。日志计时粒度为 10 ms，窗口内的最大值由冷启动样本主导。这只覆盖静止首页，实体触摸与复杂交互的帧率未在本轮测量。
[11][13]

Agent 时延与已有镜像

表 8 独立样本与文件测量

项目	结果	测量说明
历史无屏 ask	115 s; 6 轮; 6 次工具	2026-08-30 单次日报请求，原始行见会话日志；不是平均时延，也不是带屏数据。[11][12][31]
nuttx.bin	1,233,724 B	2026-09-14 已有二进制文件；未在本轮重新构建。[11]
nuttx.hex	3,470,222 B	同批 Intel HEX 文本大小；不等于 Flash 有效负载。[11]
qspi_bootstub.hex	2,076 B	2026-09-12 已有 HEX 文件大小，不能直接等同启动代码占用。[11]

该记录的文件名为 `daily-20260228.md`，验收发生在 2026-08-30，当时板端尚未做时间同步；2026-09-14 起板端已能从 eMMC 恢复时间并在线校时，见表 6。[11][12][22]

可靠性与稳定性

2026-09-14 16:16 至 21:17 连续串口观察约 5 小时，期间 11 次校时成功、无复位；页面对象与监听器计数在约 80 秒静止窗口内保持不变。原始堆统计出现负数或超出板载容量的 `free` 值，本报告不引用堆余量数据。[11][22]

以下项目本轮未测：24 小时连续运行、实物告警注入的命中率与误报率、物理拔插恢复时延、功耗、实体触摸帧率。本地规则、失败窗口及告警恢复代码提供确定性处理路径；传感器异常、接线故障和整机失效风险仍需按上述测试逐项验证。

3.6 AI-Native 开发说明

表 9 AI-Native 开发记录

指标	数据
AI Coding 代码占比	约 85%（团队估算）。范围为专属仓自研 C、头文件与脚本，本次清点 166 个文件、32,120 个物理行，含空行与注释；排除公共树、nanoMODBUS、LVGL 库和生成字体。比例不是逐行自动归因结果。
使用的 AI 工具	Claude Code、Codex、OpenCode、Cursor、Grok Build、ZCode。以 logs 中的真实 tool 标签归类，不把其他工具会话改标为官方支持来源。
MCP 工具使用情况	设备端未使用 MCP。开发侧未使用 VelaJS MCP 或 Figma MCP；不把开发工具连接能力计作设备端功能。
Skills 使用与新增情况	开发侧使用官方构建、Kconfig、驱动审查与日志采集等 Skill；自建板端内环、候选确认、告警上屏、硬件核对与 MThings 配置 Skill。设备端另有告警解释、运行报告、Modbus 查数三份运行时 Skill，见表 5。
Token 使用总量	开发阶段未使用 MiMo Token；开发工具总 Token 尚未统一汇总。板端推理使用赛事 MiMo，控制台累计用量尚未导出，不填写推测数字。

2026-09-15 证据快照记录开发会话文件 270 份，按真实来源分布为 claude-code 64；codex 64；cursor 60；grok-build 21；opencode 50；zcode 11。这些会话日志本身也是板端联调与验收的原始记录，正文按日期引用到具体文件，例如 2026-08-30 的日报验收；日志位于 logs/Foleaf/，提交前由官方校验脚本检查。[\[11\]](#)[\[17\]](#)[\[31\]](#)

开发方式与经验

AI 辅助覆盖驱动和应用实现、Kconfig 排查、主机测试、板测脚本与文档整理。开发者负责确定方案、确认配置变更和判断验收结果。项目没有对照实验去量化节省的工时，所以只说明使用方式，不给出效率提升百分比。

有一项具体经验来自 HMI 与 Agent 的内存协同。TLS、请求组装与工具分发在同一轮次里，带屏栈预算不能只按普通 UI 任务估计。相关配置与故障记录进入公共仓 PR，并将编译、烧录、串口验收与人工确认分别沉淀为开发 Skill。编译成功之后仍需真机验证，不能让日志或脚本中的预期输出替代实际结果。[15][18][19]

3.7 总结与展望

成果总结

VelaGuard 已经有一套以现场网关为中心的 openvela 固件，本地采集与规则、候选点表操作、LVGL 显示、设备 Skill 和联网推理接口都有对应入口。同板图形与 Agent 自启、历史板端多轮日报和静止界面优化都有带测试条件的证据；8 个公共仓 PR 将底层工作留在相应源码树中。2026-09-14 又补上三项板验：MQTT 四主题上报（PASS=4 FAIL=0）、eMMC 时间恢复与 SNTP 校时、32 点位点表上限；界面侧新增趋势页与多行告警页，支持逐点静音与标记处理。配套的 Windows 点表上位机与只读云看板已独立可用。[20][21][22]

应用前景与商业价值

这套方案面向小型设备现场的值守人员、嵌入式工程师和自动化集成商。使用偏好是本地可视、配置可追溯、无需长期连接电脑；适用人群按运维岗位与任务划分，无特定年龄或地区限制。云看板把单台网关扩展成多设备机队视图：设备只上报，历史与告警留在云端 SQLite，运维不必逐台上门；上位机把点表交付从命令行提前到图形界面，缩短现场调试时间。潜在交付方式是网关硬件配合配置与集成服务，云看板作为多设备管理入口。商业模式仍待现场试点验证，不作收入或市场规模预测。[28][29][30]

不足与后续重点

带屏固件的含工具 ReAct 轮次仍有 HardFault 记录，最后一次在 2026-09-14 15:19，其后约 5 小时连续观察无复位，32 KiB 栈配置后仍不能直接宣称稳定。事件主动解释、定时 Agent 日报和解释文件自动上屏尚未完成完整验收；当前运行报告是固件统计结果。[\[3\]\[15\]\[19\]\[24\]](#)

安全边界要继续补强。vgstats/vgnet 等命令要细化到子命令；文件写入范围与配置文件保护需要继续收紧；试读结果与生效版本的强绑定、AI 输出 schema 校验和 AI 推测上屏标注尚未完整落实。其中 schema 指输出字段和类型必须符合预先规定的结构。[\[2\]\[3\]\[19\]](#)

ESP-01S 备用链路的代码路径已实现，2026-09-14 板测记录中 Wi-Fi 未关联，链路切换行为需要单独验证。表 3 给出的是方案级关键器件，不作为量产定型物料清单。

24 小时稳定性、物理告警注入及恢复测试仍待完成。明文 MQTT 和测试 Broker 只按赛期演示定位；OTA、周报、阶段 2 规则库、Bridge、语音、屏幕点表编辑不计入当前交付能力。

实现与证据索引

以下编号对应正文中的可点击引用。代码与记录位于专属仓，公共框架改动通过表 4 的 PR 查阅；版本条件、原始记录位置 and 文件哈希另见 [\[11\]](#)。

[1] 本地告警判定	[2] 候选点表与人工确认	[3] 板端采集与后台文件任务
[4] 设备 Skill 与心跳内容	[5] 本次上电运行统计与报告	[6] 网络管理与链路切换
[7] 凭据配置与加密存储入口	[8] MQTT 会话与四类上报	[9] 扩展板方案与接线
[10] NSH 点表配置协议	[11] 原始记录节选与文件哈希	[12] 2026-08-30 Agent 验收记录
[13] 2026-09-13 HMI 前后对照	[14] 2026-09-13 主机与无头测试记录	[15] 带工具轮次故障记录
[16] 作品主线构建脚本	[17] 开发日志目录	[18] 开发侧可复用 Skills
[19] ai_agent 带屏适配 PR #32	[20] MQTT 四主题上报合同	[21] 2026-09-14 板端 MQTT 验收记录
[22] 2026-09-14 板端时间同步与连续观察记录		[23] 主机测试入口与用例清单
[24] 2026-09-14 带屏工具轮次故障转储		

[25] 趋势页实现

[26] 告警页多行与逐点操作

[27] MQTT 载荷构造与设备标识

[28] 云看板（只读 MQTT 看板）

[29] 云看板在线地址

[30] 点表配置上位机

[31] 2026-08-30 板端 Agent 日报会话日志